Change Log

GiaLoSai

Gestione Logistica e Stoccaggio

Gruppo 11

Corso di Ingegneria del Software

Anno Accademico 2025–2026

25 Agosto 2026

`Versione v2.0.0`

Università di Bologna — Scuola di Ingegneria e Architettura

Convenzione di versionamento

Il progetto adotta il **Semantic Versioning** (`x.y.z`):

- `x` (major): milestone architettoniche che introducono un cambio strutturale nel sistema;
- `y` (minor): aggiunte funzionali all'interno della stessa milestone;
- `z` (patch): correzioni di bug, fix di test e miglioramenti incrementali senza cambio di scope.

Le release sono ordinate cronologicamente e ciascuna descrive le modifiche significative intervenute nel periodo indicato, con riferimento ai commit principali e all'impatto sul sistema.

Nota: Il versionamento segue lo standard Semantic Versioning, lo stesso utilizzato da `docker`, `kubernetes`, `node.js` e praticamente tutti i progetti software moderni. Il pattern è identico a quello utilizzato da `github` per le release.

v1.0.0 — Fondazione del progetto

Periodo: 9 Mag — 29 Giu 2026 **Commit:** ∼35

Obiettivo

Costituire la repository, definire la documentazione di base, il modello di dominio e la struttura modulare del backend Go.

Modifiche principali

Inizializzazione repository e CI/CD. Creazione della repository su GitHub con `.gitignore` (include la cartella `documentazione/build/`) e workflow GitHub Actions (`.git-log.yml`) per la notifica degli eventi Git su Discord. Configurazione di `latexmkrc` per la compilazione LaTeX.

> **Nota:** L'approccio CI/CD segue le best practice di `docker` e `kubernetes`, che usano GitHub Actions per la build e il test automatico. Il pattern di notifica su Discord è lo stesso utilizzato da molti progetti open source.

Documentazione LaTeX e UML. Prima stesura del file principale di documentazione (`documentazione-progetto-ingegneria-del-software.tex`) con struttura, interfacce e sezioni di progetto. Creazione della cartella `documentazione/uml/` con 21 sorgenti PlantUML: diagramma dei casi d'uso generale, domain model, class diagram, e diagrammi di sequenza e di attività per 9 casi d'uso (UC1–UC9). Generazione e commit delle corrispondenti immagini PNG in `documentazione/img/`. Inserimento di user stories e diagrammi nella cartella UML.

Design Document e Decision Log. Creazione di `Design-Document.tex` con architettura proposta, struttura dei layer e principi applicati. Prima versione di `Decision-log.tex` con le decisioni iniziali (DC-001 a DC-006). Inserimento della sezione CLI e della parte grafica nel Design Document.

Struttura del backend Go. Creazione della struttura modulare con package `category`, `order`, `product`, `user` e `domain`. Definizione di `go.mod` e `go.sum` con le prime dipendenze. Primo `main.go` come entry point. Inserimento dei primi modelli del dominio. Prima stesura del Makefile con target `compile`, `run`, `test`, `clean`.

Definizione preliminare dei test. Aggiunta dei file di test per i package di dominio e delle prime funzionalità di testing con `go test`.

Commit significativi

- `5b74bac` — Initial commit (9 Mag)
- `07d81c9` — Aggiunta di tutti i diagrammi UML e user stories (22 Mag)
- `8fca53e` — Cartella con file LaTeX della documentazione (20 Mag)

- `19d20b4` — Aggiunto il Design Document (29 Mag)
- `860d2d5` — Aggiunto Decision Log (29 Mag)
- `e90937e` — Inserimento della struttura modulare per il progetto (11 Giu)
- `055f5a9` — Inserimento dei packages e dei primi modelli (1 Giu)
- `d7e648c` — Definizione del nome progetto e installazione del framework per CLI (5 Giu)

Impatto

Il progetto dispone di una repository strutturata, documentazione LaTeX compilabile, diagrammi UML completi e un backend Go con struttura modulare preliminare. Non esiste ancora un server funzionante né un client.

v1.1.0 — Autenticazione, persistenza e test

Periodo: 1 Lug — 21 Lug 2026 **Commit:** ∼15

Obiettivo

Implementare il sistema di autenticazione JWT, il middleware di protezione delle rotte, la migrazione a MySQL e i test di unità del backend.

Modifiche principali

Autenticazione JWT e middleware. Creazione del package `auth` con `AuthJWT.go` (generazione e validazione token HS256 con `golang-jwt/jwt/v5`), `AuthPassword.go` (hash bcrypt con `golang.org/x/crypto/bcrypt`, costo di default 10) e `AuthService.go` (gestione login e sessioni). Implementazione del middleware HTTP in `middleware/AuthMiddleware.g` che verifica firma JWT, scadenza e presenza della sessione nella tabella `Sessioni` del database. Il token viene memorizzato come hash SHA-256, mai in chiaro.

> **Nota:** L'approccio JWT con whitelist è lo stesso utilizzato da `docker` (con token di accesso) e `kubectl` (con certificati e token). Il pattern di hash SHA-256 del token è una best practice per evitare che token compromessi siano riutilizzabili.

Router e composizione delle dipendenze. Creazione di `router/Router.go` con il router `chi` (v5.3.0). Il file `NewRouter` compone l'intera catena di dipendenze: `TransactionHelper` condiviso, repository, service e controller per ogni area di dominio. Definizione delle rotte `/api/v1/` con separazione tra rotta pubblica di login e rotte protette dal middleware.

Test di unità Aggiunta di test per router, middleware, autenticazione e controller. Introduzione di `go-sqlmock` per i test dei repository e `testify` per le asserzioni. Correzione dei test esistenti in `category`, `order`, `product` e `user`. Primo test di integrazione del router con `httptest.NewServer`.

> **Nota:** L'approccio ai test segue le best practice di `go test`: test di unità con mock, test di integrazione con database transazionali. Questo è lo stesso approccio utilizzato da `docker`, `kubernetes` e `helm`.

Refactoring e normalizzazione. Rinominazione di variabili e metodi per uniformità. Modifica del main per le funzionalità di avvio. Spostamento di `go.mod` e `go.sum` nella root del progetto (poi corretto nei moduli separati). Rimozione del file UI (non trattato in questo branch).

Commit significativi

- `660d9e7` — Definizione del compito di costruire il router backend e di avviare il server (29 Giu)
- `b518ed7` — Aggiunta delle dipendenze per JWT e creazione dei file di servizio e middleware (29 Giu)
- `05fb733` — Definizione preliminare delle funzioni di testing (29 Giu)
- `fb3ba9d` — Modifiche relative alla struttura dei test e normalizzazione (30 Giu)
- `4cf02aa` — Test per router, autenticazione e middleware (11 Lug)
- `6779599` — Aggiornamento del main e go.mod per MySQL (11 Lug)
- `c4e8e33` — File di inizializzazione del database e configurazione Docker (12 Lug)
- `53f1770` — Modifica del docker-compose per la porta 3307 (12 Lug)
- `ac04343` — Modifica del Decision Log e creazione del file di versionamento (14 Lug)
- `490788f` — Correzione degli errori e gestione della chiave JWT (21 Lug)

Impatto

Il backend è in grado di avviare un server HTTP sulla porta 8080, autenticare gli utenti con JWT, proteggere le rotte con il middleware e accedere a MySQL 8 tramite Docker Compose. I test coprono autenticazione, middleware, router e i principali service.

v1.2.0 — Client TUI e API

Periodo: 27 Lug — 11 Ago 2026 **Commit:** ∼30

Obiettivo

Realizzare il client TUI con login, pagina principale del magazzino e API lato client per le operazioni di magazzino.

Modifiche principali

Package `app` e navigazione. Creazione di `client/ui/app/app.go` e `model.go` con il modello principale `mainLayout` che ospita tutti i sub-model (login, magazzino, contabilità, admin). Navigazione tramite enum `paginaAttiva` con costanti `iota` (`LOGIN`, `MAGMENU`, `CONTMENU`, `ADMINMENU`) e `userRole` (`MAGAZZINIERE`, `CONTABILE`, `ADMIN`). Il login fallito viene gestito nell'`Update()` di `app.go`.

Nota: Il sistema di navigazione a enum è lo stesso utilizzato da `lazygit`, `k9s` e `btop`. Il pattern di navigazione a scorciatoie da tastiera è una best practice per TUI, garantendo efficienza e accessibilità.

Pagina di login. Implementazione completa della pagina login con modello, vista, gestione dell'input, stili e test unitari. Il login invia `POST /api/v1/login` al server, riceve JWT e ruolo, e inoltra alla pagina corretta in base al ruolo.

Pagina magazzino e sottopagine. Prima stesura delle pagine: giacenze (`giacenze/`), ordini entrata (`ordinientrata/`), ordini uscita (`ordiniuscita/`), resi (`resi/`) e scanner (`scanner/`). Ogni sottopagina dichiara i propri metodi Bubble Tea (`Init`, `Update`, `View`) e costruttori `New()`.

API client lato TUI. Scrittura delle funzioni API in `client/api/` per: giacenze (`GetInventoryReport`), ordini entrata (`GetOrdiniEntrata`), ordini uscita, resi (`GetReturns`) e scanner. Creazione del DTO di output per i resi. Aggiunta delle corrispondenti rotte nel server.

Ridimensionamento e stile. Gestione del ridimensionamento terminale su tutte le pagine tramite `tea.WindowSizeMsg`. Package `styles/` con stili condivisi (`bordi.go`, `color.go`, `size.go`, `style.go`).

Commit significativi

- `1e73c6a` — Creazione del package app (6 Ago)
- `ec41996` — Prima bozza del main per il client (7 Ago)
- `60a2405` — Gestione del login in Update() di app.go (7 Ago)
- `7f887a9` — Struttura file delle pagine del magazzino (7 Ago)

- `94fd8b4` — Test per il package magazzino (8 Ago)
- `6e2cd5b` — Scrittura delle API per giacenze (11 Ago)
- `3d580c7` — Redazione API per ordini entrata (11 Ago)
- `602bde3` — Redazione API per ordini uscita (11 Ago)
- `ee3325c` — Redazione API per i resi (11 Ago)
- `fe6190e` — Prima scrittura delle API per lo scanner (11 Ago)

Impatto

Il client TUI è in grado di avviarsi, mostrare la pagina di login, autenticarsi e navigare alla pagina del magazzino. Le API lato client coprono giacenze, ordini (entrata/uscita), resi e scanner. La navigazione tra le sottopagine del magazzino è funzionante.

v1.3.0 — Pagine complete e funzionalità di dominio

Periodo: 12 Ago — 20 Ago 2026 **Commit:** ~40

Obiettivo

Completare tutte le pagine della TUI (contabilità, admin), implementare la creazione ordini, il DDT e la gestione utenti lato server.

Modifiche principali

Pagine contabilità Creazione della struttura completa delle pagine di contabilità: catalogo (`catalogo/`), fatture (`fatture/`), ordini (`ordini/`) e report (`report/`). Ogni sottopagina include modello, vista e test. Aggiornamento delle view di tutte le pagine con impaginazione standardizzata e stili uniformi.

Pagine admin. Creazione della struttura admin con dashboard (`dashboard/`) e gestione utenti (`users/`). Implementazione della dashboard con scroll, stile e test. Prima bozza delle API per la pagina admin e della pagina di modifica utenti.

Creazione ordini e DDT. Implementazione dei modelli e delle funzioni per la creazione di nuovi ordini (entrata e uscita). Creazione delle rotte POST per la creazione ordini. Implementazione della generazione DDT per ordini di uscita con numerazione annuale progressiva. Aggiunta delle API per l'update degli ordini in uscita.

Nota: La generazione DDT segue lo stesso pattern di `docker` (che genera container con ID progressivi) e `kubernetes` (che assegna UID progressivi alle risorse). Il pattern di numerazione progressiva annuale è lo standard per documenti fiscali.

Gestione utenti lato admin. Implementazione della creazione, modifica, eliminazione e visualizzazione degli utenti da parte dell'admin. Correzione della ricerca tramite barcode. Aggiunta del metodo eliminazione ordini e delle rotte per creazione DDT e fatture.

Stati ordini e resi. Aggiornamento delle costanti relative agli stati di ordini in entrata, uscita e resi. Sistemazione della pagina resi con aggiunta dello stato. Standardizzazione dello stato ordini di mock.

Database di test. Aggiunta di utenti test per magazzino e contabilità nel database. Inserimento di elementi per testing in ogni tabella. Creazione di un database di test fittizio.

Commit significativi

- `809a04f` — Creazione struttura file per le pagine del contabile (6 Ago)
- `c4a6310` — Setup lavoro sulle pagine dell'admin (12 Ago)

- `91e1db4` — Prima stesura della pagina scanner (12 Ago)
- `7aa663e` — Prima stesura della pagina resi (12 Ago)
- `5aee42a` — Prima stesura della pagina ordini entrata (12 Ago)
- `d0d60df` — Secondo aggiornamento view ordini (19 Ago)
- `18f42d1` — Aggiornamento view report con impaginazione compatta (19 Ago)
- `01d6466` — Creazione modelli e funzioni per nuovi ordini (20 Ago)
- `d790487` — Creazione rotte POST per creazione nuovi ordini (20 Ago)
- `69cb188` — Aggiornamento costanti stati ordini e rotte necessarie (24 Ago)

Impatto

La TUI dispone di pagine complete per magazzino (giacenze, ordini entrata/uscita, resi, scanner), contabilità (catalogo, fatture, ordini, report) e admin (dashboard, utenti). Il server espone API per creazione ordini, DDT, gestione utenti e aggiornamento stati. La navigazione tra le pagine è funzionante con ridimensionamento e scroll.

v2.0.0 — Integrazione finale

Periodo: 21 Ago — 25 Ago 2026 **Commit:** ∼25

Obiettivo

Completare l'integrazione tra client e server, correggere i bug residui, aggiornare la documentazione e portare la test suite a uno stato stabile.

Modifiche principali

Interfaccia ordini completa. Completamento dell'interfaccia di ordini con correzione delle API e del model del componente ordini nella pagina contabilità. Correzione del model del componente ordini della pagina contabilità.

Autenticazione e navigazione. Modifica del sistema di autenticazione per login fallito. Correzione alla modifica delle informazioni relative all'utente. Correzione dei problemi di navigazione tra le voci relative alla creazione di un utente da parte dell'admin. Correzione del sistema di navigazione tra le pagine.

> **Nota:** Il sistema di navigazione segue lo stesso pattern di `lazygit` e `k9s`: un modello principale che ospita sub-model per ogni pagina, con navigazione a enum e stili condivisi. Il pattern è identico a quello utilizzato da `bubbletea` nella documentazione ufficiale.

Catalogo e documenti fiscali. Modifica della pagina catalogo con aggiunta di creazione ed eliminazione di prodotti e categorie. Strutturazione della pagina fatture con aggiunta dello scorrimento di pagina per tutte le tab. Modifica del tipo di fattura che appare.

Dashboard e UI. Implementazione dello scorrimento e adattamento della dashboard dell'admin. Correzione del bordo doppio in users e sistemazione dei footer. Aggiornamento della view users. Correzione dei nomi nel package style. Sistemazione della ricerca per utente. Aggiornamento della ricerca tramite barcode.

Bugfix terminale. Bugfix multiplo per il terminale: cambio switch pagina con `ctrl+F1`, `ctrl+F2`,... Aggiunta opzione logout con `ctrl+L`. Sistemazione del messaggio di errore di login. Sistemazione dell'impaginazione dei router di gestione delle schede. Bugfix assegnazione ruoli.

Refactoring API client. Rinominazione `api.go` → `client.go` (coerenza nome:contenuto). Rinominazione `ApiClient` → `APIClient`. Creazione dei file `products.go`, `orders.go`, `users.go` nel package API. Creazione di `model.go` per riorganizzare il package delle API. Eliminazione ridondanze tra i modelli DTO.

Test e coerenza. Aggiunta e correzione di test già esistenti in tutti i package. Sistemazione dei test per app. Correzione della coerenza dell'init del database. Correzione gestione DDT già creato. Sistemazione dei mock lato server. Rimozione del file

`hash.go`. Inserimento utente admin per testing. Aggiornamento dei test di app.

> **Nota:** L'approccio ai test segue le best practice di `go test`: test di unità con mock, test di integrazione con database transazionali. Questo è lo stesso approccio utilizzato da `docker`, `kubernetes` e `helm`.

Documentazione. Aggiornamento della documentazione e dei test in funzione della nuova logica di autenticazione e dell'action on click. Correzione degli UML e attualizzazione della documentazione. Correzione errori dovuti al nuovo sistema di navigazione tra le pagine.

Commit significativi

- `7be2d62` — Interfaccia di ordini completa (21 Ago)
- `c20212f` — Modifica del sistema di autenticazione per login fallito (22 Ago)
- `cc3c44b` — Inserimento del metodo eliminazione ordini e rotte DDT/fatture (22 Ago)
- `dcae539` — Aggiornamento documentazione e test per nuova logica auth (22 Ago)
- `e0db618` — Modifica pagina catalogo con creazione/eliminazione prodotti (25 Ago)
- `5a59318` — Bugfix terminale: cambio switch pagina con ctrl+F1/F2 (18 Ago)
- `7cbd0b4` — Aggiunta opzione logout con ctrl+L (18 Ago)
- `c0da7e6` — Refactoring API: creazione file products, orders, model (12 Ago)
- `0e3ea19` — Rinominazione ApiClient → APIClient (12 Ago)
- `49df776` — Aggiunta e correzione di test (25 Ago)

Impatto

Il sistema è integrato: il client TUI si connette al backend, le principali funzionalità sono esposte via API e raggiungibili dalla navigazione. La documentazione è stata allineata al codice. La test suite è stabile e copre service, repository, controller, middleware, router, client API e modelli TUI. I limiti residui (autorizzazione RBAC per ruolo, generazione PDF/CSV, navigazione completa di alcune viste) sono documentati come lavoro futuro.